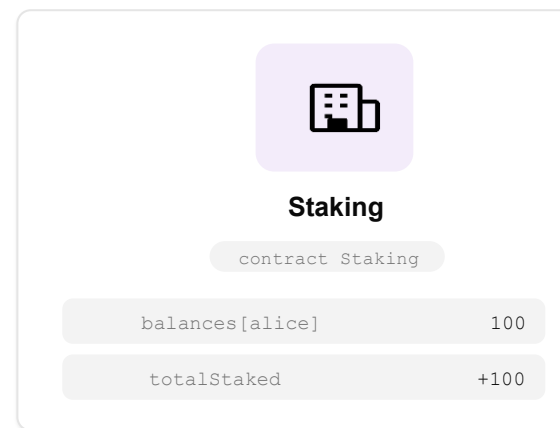
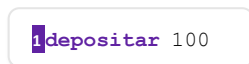
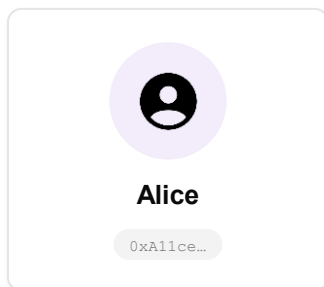


¿Por qué un ERC20 necesita approve?

Y por qué con **ETH nativo** bastaba una sola función `payable`.

Queremos un contrato de **staking**

Alice deposita fondos. El contrato debe **recibir el dinero** y **registrar el saldo interno de Alice** — todo en una sola transacción.



⚙️ **Atómico** — recibir fondos + actualizar saldo en **una sola** transacción

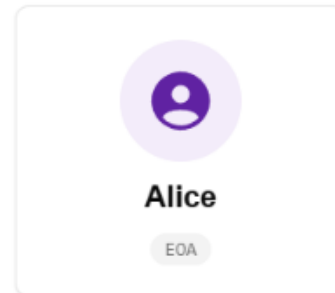
Con ETH basta una función **payable**

El ETH viaja **dentro** de la misma llamada. `msg.value` te dice cuánto llegó — actualizas el saldo en la misma función. Atómico.

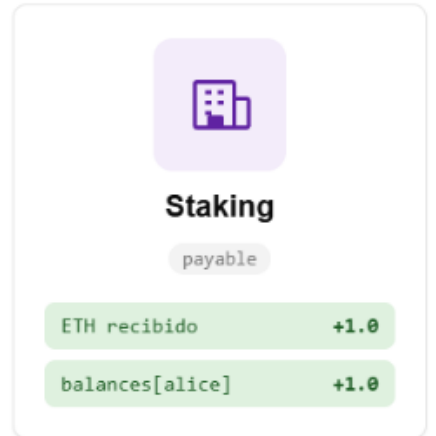
```
contract Staking {  
    mapping(address => uint256) public balances;  
  
    // 📌 payable = puede recibir ETH  
    function stake() external payable {  
        balances[msg.sender] += msg.value;  
    }  
}
```

STAKING.SOL

✓ Atómico ⚡ 1 transacción El ETH y la lógica viajan juntos.



1 stake() · value: 1 ETH



✓ Una sola llamada · El contrato controla el ETH desde el momento en que llega

Un ERC20 **no** es crypto nativa

Es solo **otro contrato** con un `mapping(address => uint256)`. No hay "monedas" que se muevan: solo entradas en una tabla que vive dentro del contrato del *token*.

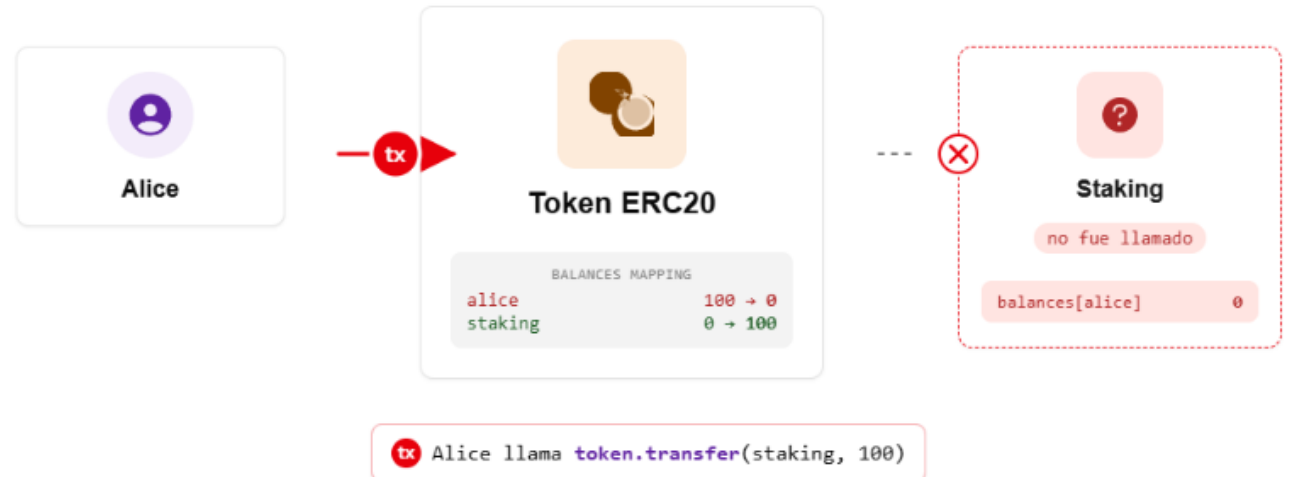


¿Y si Alice solo hace **token.transfer**?

Los tokens se mueven en el contrato del *token*... pero el contrato del staking **nunca fue llamado**. No sabe que llegaron. Resultado: fondos atascados.

```
function transfer(address to, uint256 amt) ERC20.SOL (EXTRACTO)
    external returns (bool)
{
    balances[msg.sender] -= amt;
    balances[to] += amt;
    emit Transfer(msg.sender, to, amt);

    // ⚠ se acaba aquí. No avisa al destino.
    return true;
}
```



✖ Staking nunca se ejecuta. Su `balances[alice]` sigue en 0.

⚠ Tokens "huérfanos" en la cuenta del staking — bug clásico que pierde fondos.

✖ Tokens transferidos, lógica no ejecutada — el contrato no se entera

El patrón `approve` + `transferFrom`

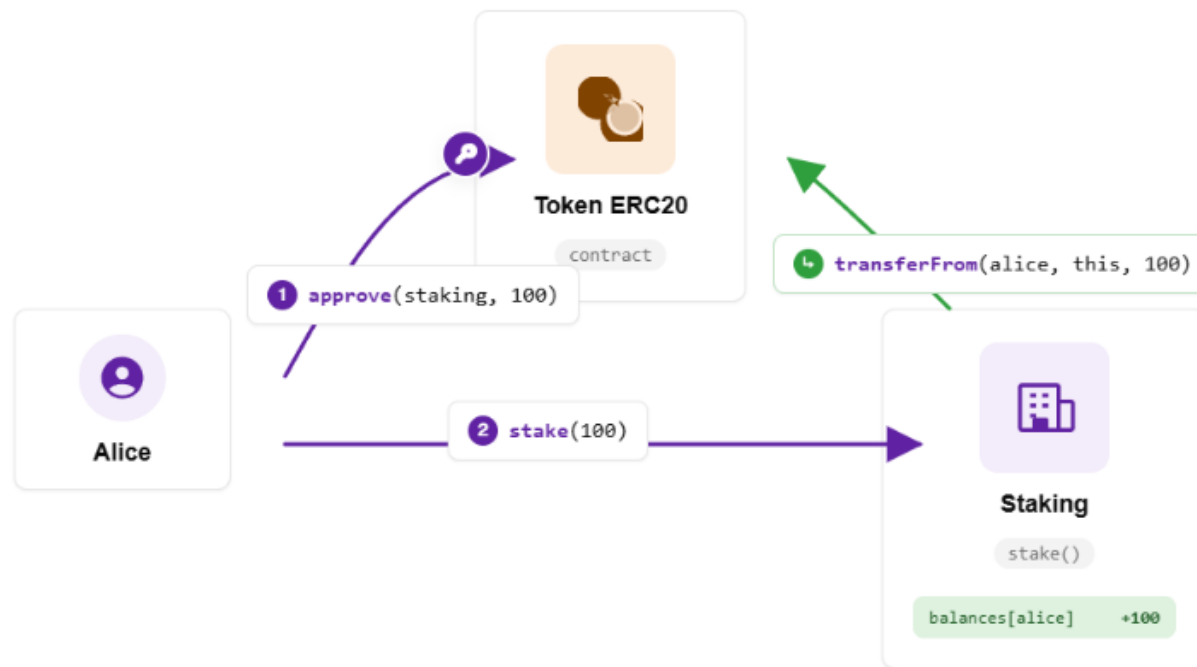
Alice **autoriza** al staking (tx 1). El staking **hace pull** dentro de su propia función (tx 2). La lógica vuelve a ser atómica.

```
// Alice autoriza al staking
token.approve(staking, 100);
// allowance[alice][staking] = 100
```

ALICE (TX 1)

```
function stake(uint256 amt) external {
  // hace pull desde dentro de la función
  token.transferFrom(msg.sender, address(this), amt);
  balances[msg.sender] += amt; // ✓ atómico
}
```

STAKING.SOL (TX 2)



✓ Atómico dentro de `stake()` — el contrato recupera el control

Tres metáforas para no olvidarlo

Push vs **pull**. Lo que cambia es *quién* ejecuta la transferencia — y eso decide si tu contrato puede reaccionar a tiempo.



ETH nativo

Carta certificada

El dinero **viaja contigo** en el sobre. Llega exactamente cuando llega tu mensaje.

```
stake() payable
// msg.value > 0
```

-
- ✓ 1 transacción
 - ✓ Atómico (push)
 - ✓ El contrato lo recibe en su función



ERC20.transfer

Paquete en la puerta

Lo dejas en la entrada y te vas. El destinatario **nunca abrió** — no sabe que llegó.

```
token.transfer(staking, 100);
// staking no se ejecuta
```

-
- ✓ 1 transacción
 - ✗ No atómico (push ciego)
 - ✗ El contrato no se entera



approve + transferFrom

Llave al destinatario

Le das una **llave acotada**. Él entra y se lleva lo justo, ejecutando su lógica al mismo tiempo.

```
token.approve(staking, 100);
staking.stake(100); // pull
```

-
- ➖ 2 transacciones
 - ✓ Atómico (pull)
 - ✓ El contrato controla la transferencia

ETH viaja con la llamada.

Los tokens hay que ir a buscarlos.

- 1 Un **ERC20** es solo otro contrato con un mapping de saldos.
No es cripto nativa, no es `msg.value`.
- 2 Los contratos **no se enteran** de cambios en otros contratos.
Si no los llamas, su lógica no corre.
- 3 **approve + transferFrom** es el patrón estándar: tu contrato **tira** de los tokens desde dentro y mantiene atomicidad.
- 4 Existen mejoras: `EIP-2612 permit`, `ERC1363`, `Permit2`.
Todas existen **porque approve existe**.

Repaso del día 11

16 preguntas sobre certificados X.509, CA vs MSP, validación local, políticas y MVCC

Las 16 preguntas — Repaso día 11

1. ¿Qué es exactamente un certificado X.509 y por qué Fabric lo eligió en vez de inventar algo propio?
2. ¿Qué información lleva dentro un certificado X.509 y cuál de esos campos es la que de verdad garantiza su validez?
3. ¿Qué es la cadena de confianza en PKI y por qué casi nunca presentamos un certificado emitido directamente por una root CA?
4. El certificado FNMT (CERES), el DNle o el candado del navegador son X.509. ¿Por qué insistimos en este paralelismo cuando explicamos Fabric?
5. Si el root CA basta para verificar la autenticidad de un certificado, ¿para qué necesita Fabric un MSP además?
6. Cuando un peer recibe una transacción firmada, ¿llama a la CA de la organización emisora para validarla?
7. Si un certificado tiene firma válida y aún no ha expirado, ¿es siempre aceptado?
8. Un certificado X.509 puede llevar codificado el rol (admin, client, peer, orderer). ¿Cómo se utiliza eso en Fabric y qué pasaría si no se usara?
9. La endorsement policy se evalúa primero en el cliente y luego otra vez por cada peer cuando llega el bloque. ¿No es redundante?
10. Las channel policies definen quién puede modificar la configuración del canal. ¿Cuándo se aplican exactamente y por qué requieren firmas múltiples?
11. Si las endorsement policies ya controlan qué transacciones valen, ¿para qué sirven además las ACLs en Fabric?
12. ¿Dónde se almacena la configuración de un canal (organizaciones, MSPs, policies) y por qué eso es relevante?
13. El orderer no ejecuta chaincode ni mantiene el world state. ¿Por qué guarda entonces una copia del ledger de cada canal?
14. Multi-Version Concurrency Control (MVCC) es lo que comprueba Fabric en la fase Validate. ¿Qué problema concreto resuelve?
15. Una aplicación cliente quiere invocar chaincode pero su organización aún no ha emitido un certificado para ella. ¿Qué ocurre?
16. El certificado de un peer va a expirar la semana que viene. ¿Qué pasos hay que dar y se ve afectado el canal?

Qué es un certificado X.509

Pregunta

¿Qué es exactamente un certificado X.509 y por qué Fabric lo eligió en vez de inventar algo propio?

Respuesta razonada

Es un documento digital firmado por una autoridad de confianza (CA) que vincula una identidad con una clave pública. Es un estándar ITU-T (RFC 5280) anterior a internet, ampliamente probado durante décadas: lo usa HTTPS, el DNle, los certificados FNMT, las apps firmadas... Fabric no inventa nada nuevo en identidad: reusa una pieza madura que la gente ya conoce y que tiene herramientas, librerías y operadores expertos detrás.

Qué contiene un certificado X.509

Pregunta

¿Qué información lleva dentro un certificado X.509 y cuál de esos campos es la que de verdad garantiza su validez?

Respuesta razonada

Contiene sujeto (a quién pertenece), clave pública del sujeto, emisor (la CA que firmó), validez (desde/hasta cuándo), número de serie, extensiones (usos, roles) y la firma digital del emisor. Lo que garantiza la validez es esa firma: cualquier alteración del resto rompe la firma y el certificado se invalida. Sin ella, sería texto plano que cualquiera podría modificar.

Pregunta

¿Qué es la cadena de confianza en PKI y por qué casi nunca presentamos un certificado emitido directamente por una root CA?

Respuesta razonada

Para fiarte de un certificado, te fías de quien lo firmó, y de quien firmó a este, hasta llegar a una root CA que reconoces como ancla de confianza. En la práctica las root CAs se mantienen offline y solo firman CAs intermedias que sí operan online emitiendo certificados. Así, si una intermedia se compromete, se revoca y se emite otra; la root no se toca y la confianza global del sistema se mantiene.

Pregunta

El certificado FNMT (CERES), el DNle o el candado del navegador son X.509. ¿Por qué insistimos en este paralelismo cuando explicamos Fabric?

Respuesta razonada

Porque elimina la sensación de "esto es magia blockchain". El alumno YA usa X.509 cuando hace la Renta, firma con el DNle, navega por HTTPS o instala una app firmada. La identidad en Fabric es exactamente la misma tecnología: una CA que emite, un certificado que firma, una clave pública que verifica. Lo único "nuevo" en Fabric es el MSP que decide qué CAs son de confianza dentro de cada red — el resto es PKI clásica.

Pregunta

Si el root CA basta para verificar la autenticidad de un certificado, ¿para qué necesita Fabric un MSP además?

Respuesta razonada

Porque autenticidad no es lo mismo que confianza operativa. El root CA te dice "este certificado lo emitió quien dice ser", pero no te dice si AHORA debe seguir siendo válido, si la persona tiene el rol correcto o si está revocado. El MSP añade lo que falta: lista de revocación (CRL), certificados intermedios reconocidos, mapeo de roles (admin/client/peer/orderer) y reglas de gobernanza para actualizar todo eso vía configuración del canal.

Pregunta

Quando un peer recibe una transacción firmada, ¿llama a la CA de la organización emisora para validarla?

Respuesta razonada

No. Toda la validación se hace LOCAL contra el MSP configurado en el canal. El peer toma el certificado, busca por MSP ID la organización a la que pertenece y verifica tres cosas: que la cadena de firmas lleva a una CA confiable, que el certificado no aparece en la CRL del MSP y que la firma de la transacción es correcta. Sin conexión externa. Eso es lo que permite a Fabric ser rápido y resistente a problemas de red de las CAs.

Pregunta

Si un certificado tiene firma válida y aún no ha expirado, ¿es siempre aceptado?

Respuesta razonada

No: el MSP mantiene una lista de revocación (CRL) emitida por la CA. Si el certificado aparece en ella, se rechaza aunque la firma matemática siga siendo válida. La revocación cubre el caso típico: empleado que se va, clave privada comprometida, peer dado de baja. Sin CRL en el MSP, los certificados solo expirarían por fecha y eso es demasiado lento como mecanismo de seguridad.

Pregunta

Un certificado X.509 puede llevar codificado el rol (admin, client, peer, orderer). ¿Cómo se utiliza eso en Fabric y qué pasaría si no se usara?

Respuesta razonada

Fabric usa OUs (Organizational Units) o extensiones para clasificar el portador. El MSP lee ese rol y aplica las políticas en consecuencia: solo un "admin" puede aprobar cambios de config; solo un "peer" puede endosar; un "client" solo puede enviar transacciones. Sin esa clasificación, todos los certificados serían iguales y necesitarías mantener listas manuales por rol — pesado y propenso a errores.

Pregunta

La endorsement policy se evalúa primero en el cliente y luego otra vez por cada peer cuando llega el bloque. ¿No es redundante?

Respuesta razonada

No, es defensa en profundidad. La evaluación del cliente es optimista: "ya tengo suficientes firmas, mando la tx". La evaluación en commit es la autoritativa: cada peer comprueba que las firmas cumplen la política antes de aplicar los cambios al ledger. Sin la segunda, un cliente malicioso podría enviar una tx con firmas inválidas o insuficientes esperando que cuele en el orderer. La política se aplica donde manda: en el commit.

Pregunta

Las channel policies definen quién puede modificar la configuración del canal. ¿Cuándo se aplican exactamente y por qué requieren firmas múltiples?

Respuesta razonada

Se aplican cada vez que alguien intenta cambiar la red: añadir una organización, actualizar un MSP, cambiar el orderer, etc. Requieren firmas múltiples (típicamente MAJORITY de admins) precisamente porque son cambios estructurales que afectan a todos. Sin ese requisito, un solo admin podría meter una org sin consentimiento de los demás — eso rompería el modelo de gobernanza compartida que justifica usar blockchain en lugar de una BBDD.

Pregunta

Si las endorsement policies ya controlan qué transacciones valen, ¿para qué sirven además las ACLs en Fabric?

Respuesta razonada

Las endorsement policies dicen "esta tx es válida si firman estos peers". Las ACLs son más granulares: "esta función concreta del chaincode solo puede invocarla quien tenga este rol", o "solo los admins pueden instalar contratos". Es la diferencia entre permiso de operación (ACL) y permiso de validación (endorsement). Las ACLs filtran antes; las endorsement policies validan después.

Dónde vive la configuración del canal

Pregunta

¿Dónde se almacena la configuración de un canal (organizaciones, MSPs, policies) y por qué eso es relevante?

Respuesta razonada

En config blocks dentro del propio ledger del canal, no en un fichero externo. El orderer guarda el bloque génesis (la configuración inicial), y cualquier cambio posterior aprobado por las policies del canal se añade como un nuevo config block. Es relevante porque la configuración es PARTE de la historia auditable del canal: no puede cambiarse sin dejar rastro y siempre está sincronizada con quien tenga el ledger.

Pregunta

El orderer no ejecuta chaincode ni mantiene el world state. ¿Por qué guarda entonces una copia del ledger de cada canal?

Respuesta razonada

Porque necesita la SECUENCIA OFICIAL de bloques para mantener consenso (Raft compara bloques entre orderers), para conservar la configuración del canal (que vive dentro del ledger) y para ayudar a resincronizar peers que se quedan atrás. Lo que NO guarda es el world state ni nada que dependa de ejecutar chaincode: solo la cadena de bloques en su orden canónico.

Pregunta

Multi-Version Concurrency Control (MVCC) es lo que comprueba Fabric en la fase Validate. ¿Qué problema concreto resuelve?

Respuesta razonada

El de las simulaciones obsoletas. Entre que el endorser simula una tx (leyendo el world state actual) y el commiter la aplica al ledger, otra transacción puede haber modificado los mismos datos. MVCC compara el read-set de la simulación con el world state real al hacer commit: si lo que se leyó ya no es lo que está, la tx se marca como inválida aunque las firmas sean correctas. Sin MVCC, dos tx concurrentes podrían pisarse y romper la consistencia.

Pregunta

Una aplicación cliente quiere invocar chaincode pero su organización aún no ha emitido un certificado para ella. ¿Qué ocurre?

Respuesta razonada

Simplemente no puede operar. Cualquier propuesta que enviara llegaría sin firma válida y los endorsers la rechazarían al verificar contra el MSP. La emisión del certificado por la CA es el paso 0 antes de poder hablar con la red — equivalente a que un usuario sin DNI no pueda hacer la Renta. La identidad no es opcional en Fabric: es prerequisite.

Rotación de certificado de un peer

Pregunta

El certificado de un peer va a expirar la semana que viene. ¿Qué pasos hay que dar y se ve afectado el canal?

Respuesta razonada

La CA de su org emite un nuevo certificado al peer (con la misma clave pública o una nueva), se actualiza el material en el MSP local del peer y, si la jerarquía no cambia (mismo root/intermediate CA), el MSP del canal NO necesita actualizarse — las firmas seguirán validándose porque la cadena de confianza sigue intacta. Solo hace falta tocar el canal si cambia la CA emisora o si se rota el root.